

위치 기반 서비스 Location Based Service



목차

LBS 개요

LBS 사용 준비

- ◆ Google Play Service
- ◆ Location Permissions

기기 위치 확인

Geocoding

외부 지도 앱의 사용

위치 기반 서비스 (LBS: Location Based Service) 동덕여자대학교

개요 네비게이션

- ◆ 위치 정보를 수신 받아 다른 정보와 결합하여 위치 특성을 반영한 정보를 제공하는 서비스

지도와의 결합 Point Of Interest

- ◆ 위치 정보를 특정 지도 위치에 표시
- ◆ 네비게이션: 위치 정보와 지도 정보를 결합

스마트폰의 위치 정보 확인 방법

GPS는
건물 외부

- ◆ GNSS(Global Navigation Satellite System): 위성으로부터 위치 정보 수신: GPS, Galileo, GLONASS, Beidou
 - RNSS: QZSS, KPS(2035년 예정)

네트워크는
건물 내부

- ◆ 네트워크: 모바일통신이나 Wi-Fi 연결 정보를 토대로 위치 파악
- ◆ 그 밖에 NFC, RFID, Bluetooth 또는 UWB 를 사용한 초근거리 위치 확인 기술 등이 존재

구현환경 준비 01 – Google Play Service 설정 동북여자대학교

Google Play Services 설치 확인

◆ [Tools] → [SDK Manager] → [SDK Tools] 탭

SDK Platforms <u>SDK Tools</u> SDK Update Sites		
Below are the available SDK developer tools. Once installed, the IDE will automatically check for updates to display available versions of an SDK Tool.		
Name	Version	
☐ Android Emulator	34.1.20	
☒ Android Emulator hypervisor driver (installer)	2.2.0	
☐ Android SDK Platform-Tools	35.0.1	
☐ Google Play APK Expansion library	1	
☐ Google Play Instant Development SDK	1.9.0	
☐ Google Play Licensing Library	1	
☒ Google Play services	49	
☒ Google USB Driver	13	

◆ Google Play Service

- 안드로이드 운영체제의 버전과 독립적으로 핵심 기능을 제공 및 업데이트 할 수 있도록 구성한 주요 구성요소
- 위치 서비스, 인증/결제 시스템, 보안 기능 등의 핵심 기능 제공

구현환경 준비 02 – Emulator 확인

PlayStore 사용 가능 버전의 Emulator 사용 필요

◆추가 시

Choose a device definition

Q-

Category	Name	Play Store	Size	Resolution	Density
Phone	Pixel 7		6.31"	1080x2400	420dpi
Tablet	Pixel 6a		6.13"	1080x2400	420dpi

Google Play Store 사용 가능 항목 선택

◆System Image 선택

Select a system image

Recommended x86 Images Other Images

Release Name	API Level	ABI	Target
API 34 	34	x86_64	Android API 34 (Google Play)
Tiramisu	33	x86_64	Android 13.0 (Google Play)
Sv2 	32	x86_64	Android 12L (Google Play)

Target 이 Google Play 인 Image 선택

구현환경 준비 03 - 기기에 가상 위치 생성 앱 설치

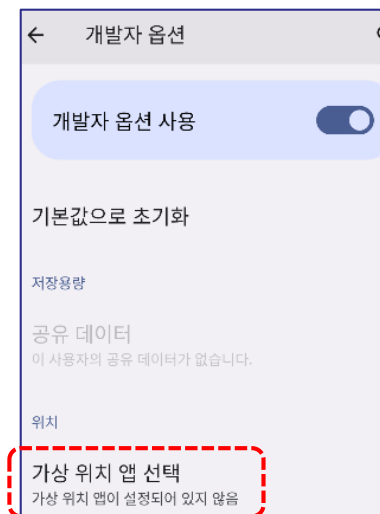
MockUp GPS 앱 설치

- ◆ Emulator 실행 후 Google Plays Store 에서 가상의 GPS 정보를 생성하는 Mockup GPS 앱 선택



Mockup GPS 정보 설정

- ◆ [개발자 옵션] 활성화
 - [설정] → [기기 정보] → [빌드 번호] 7번 클릭
- ◆ MockGPS 사용 설정
 - [설정] → [시스템] → [개발자 옵션] → [가상 위치 앱 선택]
 - 설치한 Mockup GPS 앱 지정

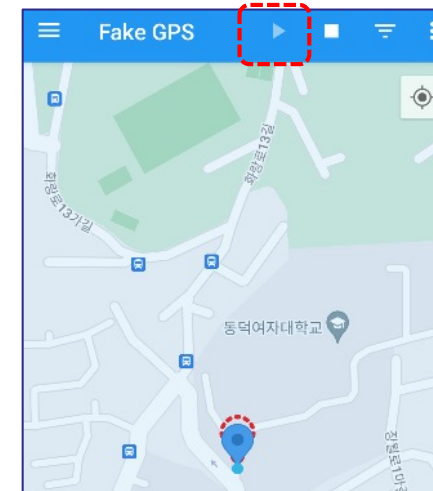
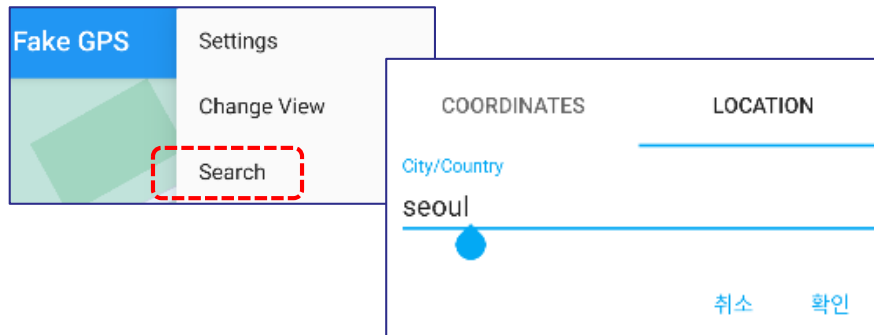


가상 위치 앱 선택
모의 위치 앱: Fake GPS

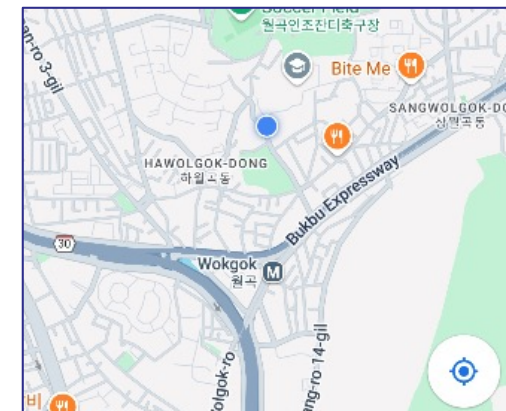
구현환경 준비 04 - 위치 확인 테스트

MockUp 위치앱 실행

- 원하는 위치 지정 후 가상위치 실행



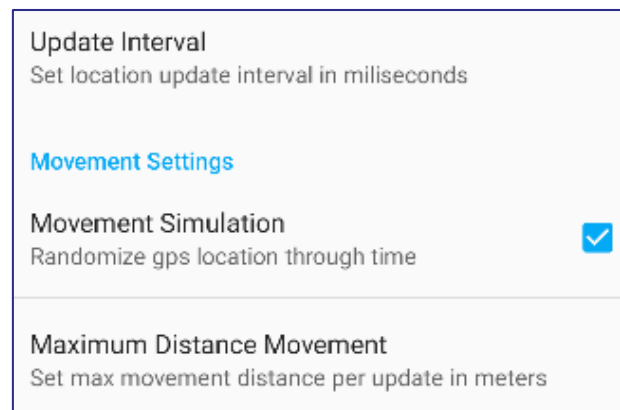
- Google Maps 등을 실행하여 위치 확인



- 앱 추가 설정

- 수신 간격
- 이동 시뮬레이션

예:
수신간격: 1000ms
이동거리: 10m



LBS 사용 준비 01

Dependency 추가: build.gradle (Module: ...)

```
// Google play service 위치 관련
implementation ("com.google.android.gms:play-services-location:21.3.0")
```

구현 시 GooglePlay 제공 클래스 import

```
import com.google.android.gms.location.*
```

위치 사용 관련 Permission 추가

- ◆ AndroidManifest에 위치 관련 Permission 추가
- ◆ 사용하고자 하는 위치 정확도에 따라 선택
 - ACCESS_COARSE_LOCATION (거리 블록 정도의 정확도 수준)
 - ACCESS_FINE_LOCATION (정확한 위치 정확도)

대략적인 위치

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

정확한 위치

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```


LBS 사용 준비 02 – Permission 실시간 요청 등덕여자대학교

Permission 실시간 요청

- ◆ AndroidManifest 에 필요 권한 기록
- ◆ 중요한 Permission의 경우 앱 실행 중 관련 권한 요청 필요 (Android 6.0 이후)
- ◆ 사용자는 앱 실행 중 권한 요구를 선택 또는 취소하고 언제든지 변경할 수 있음

실시간 Permission 확인 절차

1. 지정함수 앱 설치 시 및 필요 시 checkSelfPermission() 으로 필요 권한 확인
2. `ActivityResultContracts.RequestMultiplePermission()` 또는 `.RequestPermission()` 으로 필요 권한 요청 여러 개
3. 사용자의 응답에 따라 앱의 실행 여부 결정

LBS 사용 준비 03 – Permission 실시간 요청 등덕여자대학교

권한 확인 실행의 예

- ◆ checkSelfPermission() : 권한 확인
- ◆ requestPermissions() : 권한 요청

checkSelfPermission (권한) 결과가
PackageManager.PERMISSION_GRANTED
또는 .PERMISSION_DENIED 인지 확인

```
private fun checkPermissions() {  
    if ( checkSelfPermission(ACCESS_FINE_LOCATION)  
        == PackageManager.PERMISSION_GRANTED  
        && checkSelfPermission(ACCESS_COARSE_LOCATION)  
            == PackageManager.PERMISSION_GRANTED ) {  
        Log.d(TAG, "필요 권한 있음")  
        // 확인 후 수행해야 할 기능 실행  
    } else {  
        // 권한이 없을 경우 권한 요청  
        locationPermissionRequest.launch(  
            arrayOf(ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION)  
        )  
    }  
}
```

위치관련 기능 추가 위치

권한 요청 작업을 실행하는
ActivityResultLauncher 객체
→ registerForActivityResult() 가 생성

LBS 사용 준비 05 – Permission 실시간 요청 동덕여자대학교

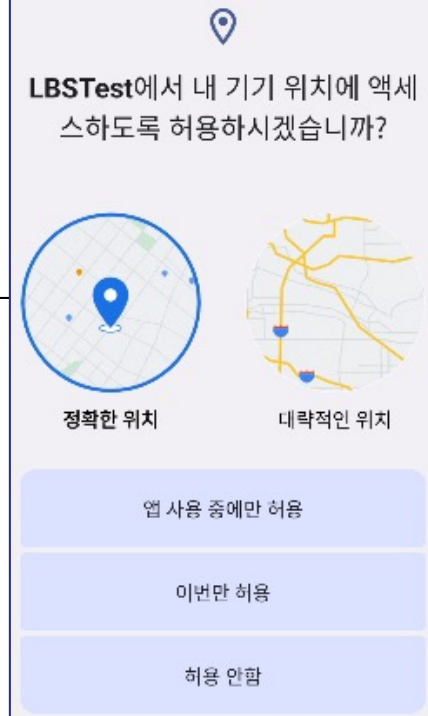
권한요청의 사용자 응답 처리

Activity 결과 요청 등록 →
ActivityResultLauncher 생성

```
val locationPermissionRequest = registerForActivityResult(  
    ActivityResultContracts.RequestMultiplePermissions() ) { permissions ->  
    when {  
        permissions.getOrDefault(ACCESS_FINE_LOCATION, false) -> {  
            // 정확한 위치 접근 권한 승인거부 후 해야할 작업  
            Log.d(TAG, "정확한 위치 사용")  
        }  
        permissions.getOrDefault(ACCESS_COARSE_LOCATION, false) -> {  
            // 근사 위치 접근 권한 승인 후 해야할 작업  
            Log.d(TAG, "근사 위치 사용")  
        }  
        else -> {  
            // 권한 미승인 시 해야 할 작업  
            Log.d(TAG, "권한 미승인")  
            flag사용해서 false저장 후  
            위치 정보 사용할 수 없도록  
        }  
    }  
}
```

Default : false

사용자 응답 처리



위치 확인 01 – 관련 클래스 및 함수

☞ 위치 확인 관련 객체 구현 및 생성

◆ FusedLocationProviderClient 객체 생성

- 위치 확인을 수행

◆ LocationRequest 구현 및 객체 생성

- 위치정보 수신 조건 지정 정보를 받는 간격

◆ LocationCallback 구현 및 객체 생성

- 위치정보 수신 후 수행해야 할 기능 구현

☞ 위치 확인 실행

- ◆ FusedLocationProviderClient 클래스의 `requestLocationUpdates(request, callback, looper)` 실행

☞ 위치 확인 종료

- ◆ `removeLocationupdates(callback)`

위치 확인 02 – 위치 서비스 클래스

FusedLocationProviderClient

- ◆ Google Play Services 에서 제공하는 위치확인 관련 클래스
- ◆ 시스템 제공 클래스인 LocationManager 역할 대체
- ◆ 최종위치 및 현재위치 확인

선언 및 생성

```
private lateinit var fusedLocationClient: FusedLocationProviderClient

override fun onCreate(savedInstanceState: Bundle?) {
    ...
    fusedLocationClient = LocationServices.getFusedLocationProviderClient(this)
    ...
}
```

Context

위치 확인 03 – 위치 요청정보 클래스

LocationRequest

- ◆ 위치정보 수신을 위한 조건 지정 : Builder(ms) 로 생성
- ◆ setIntervalMillis(ms)
 - 위치정보 업데이트 간격을 ms 로 지정
- ◆ setMinUpdateIntervalMillis(ms)
 - 위치정보 업데이트의 최소간격 지정
- ◆ setPriority(): 요청 우선순위 및 source 지정
 - PRIORITY_HIGH_ACCURACY : GPS
 - PRIORITY_BALANCED_POWER_ACCURACY : Wi-Fi 또는 Cell network

```
private lateinit var locationRequest : LocationRequest

override fun onCreate(savedInstanceState: Bundle?) {
    ...
    locationRequest = LocationRequest.Builder(3000) // 수신 간격
        .setMinUpdateIntervalMillis(5000) // 최소 수신 간격
        .setPriority(Priority.PRIORITY_BALANCED_POWER_ACCURACY) // 수신 우선순위
        .build()
}
```

위치 확인 04 – 위치 수신정보 클래스

LocationCallback 구현

- ◆ 위치정보의 수신 결과를 전달 받는 클래스
- ◆ `onLocationResult(LocationResult locationResult)`를 재정의하여 위치정보 수신 시 수행할 동작 지정
- ◆ `requestLocationUpdates()`에 전달

```
private lateinit var LocationCallback : LocationCallback

override fun onCreate(savedInstanceState: Bundle?) {
    ...
    LocationCallback = object : LocationCallback() {
        override fun onLocationResult(locationResult: LocationResult) {
            val currentLocation : Location = locationResult.locations[0]
            Log.d(TAG, "위도: ${currentLocation.latitude}, " +
                "경도: ${currentLocation.longitude}")
        }
    }
}
```

위치정보들을
보관하는 클래스

Location
: 위치정보를 표현

위치 확인 05 – 위치 수신 시작

❏ 위치정보 수신 실행 사전 확인

- ◆ 위치 제공자가 사용가능한지 확인: network 와 GPS
- ◆ 위치 관련 Permission 설정 확인
- ◆ 위치 요청관련 설정 정보 확인

❏ 위치 확인 실행 – 버튼 클릭 시 등

- ◆ requestLocationUpdates() 실행

FusedLocationProviderClient 객체

실행 전 권한확인 필요

```
private fun startLocationUpdate() {
    fusedLocationClient.requestLocationUpdates(
        locationRequest, locationCallback, Looper.getMainLooper()
    )
}
```

시스템의 메시지를
전달받는 Looper 객체

위치 확인 06 – 위치 확인 종료

위치 확인 종료

- ◆ removeLocationUpdates() 실행

FusedLocationProviderClient 객체

LocationCallback 객체

`fusedLocationClient.removeLocationUpdates(locationCallback)`

사용자에게 의해 종료가 안 될 경우 대비

- ◆ Activity 의 생명주기 함수인 onPause()에서 호출

```
override fun onPause() {
    super.onPause()
    fusedLocationClient.removeLocationUpdates(locationCallback)
}
```

위치 확인 07 – 최종 위치 확인

최종 위치의 사용

- ◆ 기기가 마지막으로 확인한 위치를 확인
- ◆ 앱 실행 시 위치정보를 수신하지 못 할 경우 기본 위치 또는 마지막 수신위치로 위치 표시

최종 위치 확인

- ◆ 각 Listener 는 Task<TResult> 객체 반환

onCreate에!

```
private fun getLastLocation() {
    fusedLocationClient.lastLocation.addOnSuccessListener { location: Location? ->
        if (location != null) {
            Log.d(TAG, location.toString())
        }
    }
    fusedLocationClient.lastLocation.addOnFailureListener { e: Exception ->
        Log.d(TAG, e.toString())
    }
}
```

getLastLocation() : Task<TResult>

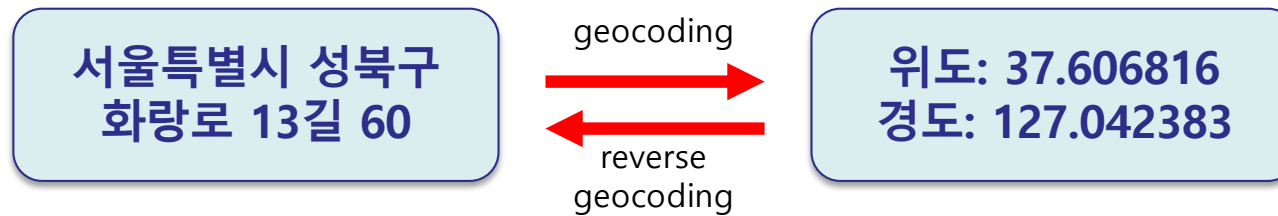
최종 위치 확인 성공 시

최종 위치 확인 실패 시

지오코딩(Geocoding) 01

개요

- ◆ Geocode: 일반 주소를 위도(latitude)/경도(longitude) 로 변환
- ◆ Reverse geocode: 위도/경도를 일반 주소로 변환



Geocoder 확인

- ◆ 안드로이드 기본 프레임워크가 아닌 Backend 서비스에 포함되어 있으므로 사용가능 가부의 확인 필요
- ◆ Geocoder.isPresent() 로 확인 → 위치 권한 필요

지오코딩(Geocoding) 02

Geocoder의 생성

- ◆Geocoder는 네트워크를 통해 정보를 획득하며, 별도의 실행흐름에서 수행 필요
- ◆Android API 33 이상 필요

기기 지정
지역

Reverse Geocoding

```
private lateinit var geocoder : Geocoder
geocoder = Geocoder(this, Locale.getDefault())
```

- ◆Geocoder.getFromLocation(위도, 경도, 개수, Geocoder.GeocoderListener)

위도

경도

결과 개수

GeocoderListener
람다 함수

```
geocoder.getFromLocation(37.505816, 127.042383, 5) { addresses ->
    CoroutineScope(Dispatchers.Main).launch {
        Log.d(TAG, addresses.get(0).getAddressLine(0).toString())
    }
}
```

외부 서버에서 정보를 가져오므로
Coroutine 적용 가능

첫 번째 Address 객체의 주소 확인

지오코딩(Geocoding) 03



Geocoding

- ◆ getFromLocationName(주소명, 개수, Geocoder.GeocoderListener)

```
geocoder.getFromLocationName("동덕여자대학교", 5 ) { addresses ->
    CoroutineScope(Dispatchers.Main).launch {
        Log.d(TAG, "위도: ${addresses.get(0).latitude}, " +
            "경도: ${addresses.get(0).longitude}")
    }
}
```

첫 번째 Address 객체의 위도/경도

지도 앱 사용하기

안드로이드 기본 지도앱 사용

```
fun callExternalMap() {

    val locLatLng // 위도/경도 정보로 지도 요청 시
        = String.format("geo:%f,%f?z=%d", 37.606320, 127.041808, 17)

    val locName // 위치명으로 지도 요청 시
        = "https://www.google.co.kr/maps/place/" + "Hawolmok-dong"

    val uri = Uri.parse(locLatLng)
    // val uri = Uri.parse(locName)

    val intent = Intent(Intent.ACTION_VIEW, uri)
    startActivity(intent)
}
```

실습

 GPS 에서 전달하는 현재 위치의 위도, 경도를 읽어
와 해당 위치의 실주소를 출력하는 앱을 구현하시오.

- ◆ GPS 사용
- ◆ 위치가 바뀔 때마다 현재의 위도/경도 확인 후 이를
Geocoding 을 통해 주소로 바꾸어 출력

 앱 종료 후 재실행 시 마지막 수신 위치 정보를 실주
소로 출력하도록 위의 앱을 수정하시오.

참고

위치 정보 처리

- ◆ <https://developer.android.com/training/location?hl=ko>

위치정보 접근권한 요청

- ◆ <https://developer.android.com/training/location/permissions?hl=ko>

Activity에서 결과 가져오기

- ◆ <https://developer.android.com/training/basics/intents/result?hl=ko>

구글맵 스키마 관련 정보

- ◆ <https://developers.google.com/maps/documentation/urls/android-intents?hl=ko>